



WPI

CS 534

Artificial Intelligence

Week 10: Natural Language Processing II

By

Ben C.K. Ngan

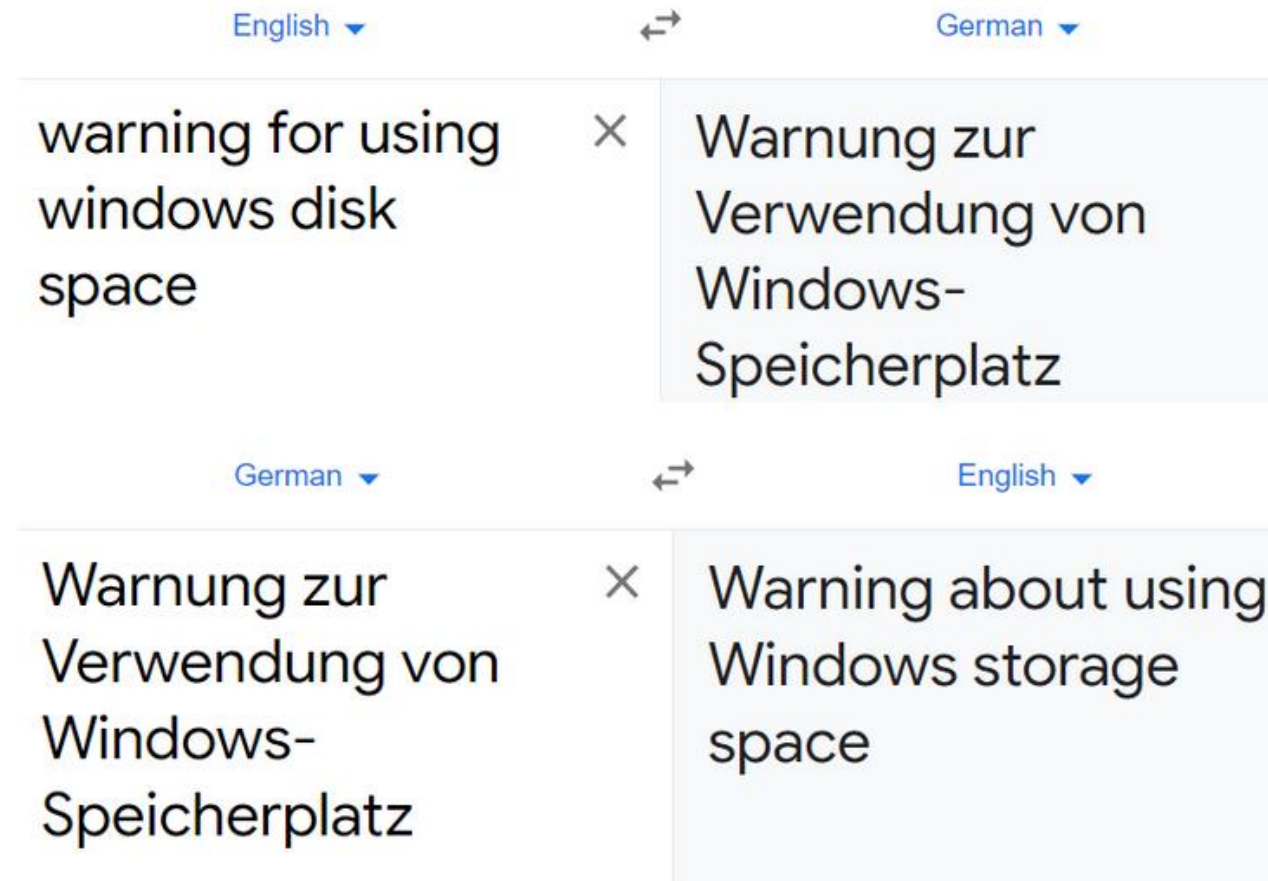
Imbalanced Data for Text Classification

- **Google Translation:**

- Expand the count of minority groups
- Translate the given sentence to 'non-English' language and then again translating to 'English'.
- Maintain the significant details of the input message but the order of words / sometimes new words with similar meaning are introduced as a new record and thus boosting the count of insufficient class.

- **Useful Resources:**

- <https://translate.google.com/>
- <https://py-googletrans.readthedocs.io/en/latest/>
- <https://github.com/NandhiniN85/Class-Imbalancing/blob/main/NLP%20-%20Class%20Imbalanced.ipynb>



Input text – “warning for using windows disk space”

Data Augmented text – “Warning about using Windows storage space”

Imbalanced Data for Text Classification

- **Class Weights:**
 - Leverage the class weights parameter during the fit model process.
 - Assign a class weight parameter for each class in the target.
 - Get more weight for the minority class when compared to the majority ones.
 - Give equal attention of the model to all the classes present in the output during the training process.
- **Useful Resources:** https://github.com/NandhiniN85/Class-Imbalancing/blob/main/English_raw200_model_execution.ipynb

```
y_integers = np.argmax(raw_y_train, axis=1)
class_weights = compute_class_weight('balanced', np.unique(y_integers), y_integers)
d_class_weights = dict(enumerate(class_weights))
```

d_class_weights

```
{0: 0.23796731587194986,
1: 3.8724954462659382,
2: 4.218253968253968,
3: 3.7102966841186737,
4: 1.3345888261142498,
5: 3.525704809286899}
```

$w_j = n_{\text{samples}} / (n_{\text{classes}} * n_{\text{samplesj}})$, where

w_j is the weight for each class(j signifies the class)

n_{samples} is the total number of samples in the dataset

n_{classes} is the total number of unique classes in the target

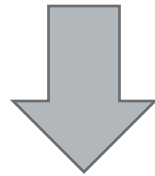
n_{samplesj} is the total number of samples of the respective class

Text Similarity

- The Jaccard coefficient can be a value between 0 and 1, with 0 indicating no overlap and 1 complete overlap between the sets.

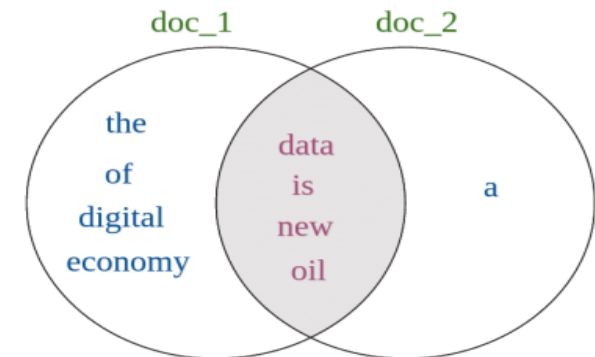
$$J(doc_1, doc_2) = \frac{doc_1 \cap doc_2}{doc_1 \cup doc_2}$$

```
doc_1 = "Data is the new oil of the digital economy"  
doc_2 = "Data is a new oil"
```



```
words_doc1 = {'data', 'is', 'the', 'new', 'oil', 'of', 'digital', 'economy'}  
words_doc2 = {'data', 'is', 'a', 'new', 'oil'}
```

$$\begin{aligned} J(doc_1, doc_2) &= \frac{\{'data', 'is', 'the', 'new', 'oil', 'of', 'digital', 'economy'\} \cap \{'data', 'is', 'a', 'new', 'oil'\}}{\{'data', 'is', 'the', 'new', 'oil', 'of', 'digital', 'economy'\} \cup \{'data', 'is', 'a', 'new', 'oil'\}} \\ &= \frac{\{'data', 'is', 'new', 'oil'\}}{\{'data', 'a', 'of', 'is', 'economy', 'the', 'new', 'digital', 'oil'\}} \\ &= \frac{4}{9} = 0.444 \end{aligned}$$



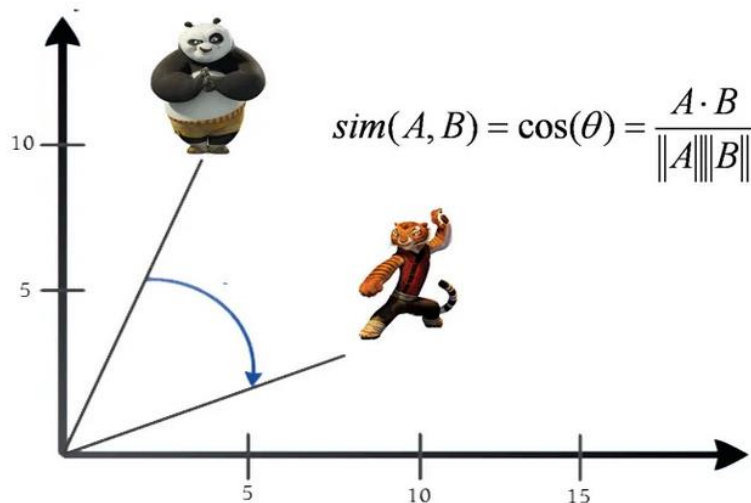
Text Similarity

- The cosine similarity is a number between 0 and 1.

$$\text{similarity}(\text{doc}_1, \text{doc}_2) = \cos(\theta) = \frac{\text{doc}_1 \cdot \text{doc}_2}{|\text{doc}_1| |\text{doc}_2|}$$

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

Cosine Similarity



```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"
```

```
# Vector representation of the document
doc_1_vector = [1, 1, 1, 1, 0, 1, 1, 2]
doc_2_vector = [1, 0, 0, 1, 1, 0, 1, 0]
```

	data	digital	economy	is	new	of	oil	the
doc_1	1	1	1	1	0	1	1	2
doc_2	1	0	0	1	1	0	1	0

$$\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n A_i B_i$$

$$= (1 * 1) + (1 * 0) + (1 * 0) + (1 * 1) + (0 * 1) + (1 * 0) + (1 * 1) + (2 * 0) \\ = 3$$

$$\sqrt{\sum_{i=1}^n A_i^2} = \sqrt{1 + 1 + 1 + 1 + 0 + 1 + 1 + 4} = \sqrt{10}$$

$$\sqrt{\sum_{i=1}^n B_i^2} = \sqrt{1 + 0 + 0 + 1 + 1 + 0 + 1 + 0} = \sqrt{4}$$

$$\text{cosine similarity} = \cos\theta = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} = \frac{3}{\sqrt{10} * \sqrt{4}} = 0.4743$$

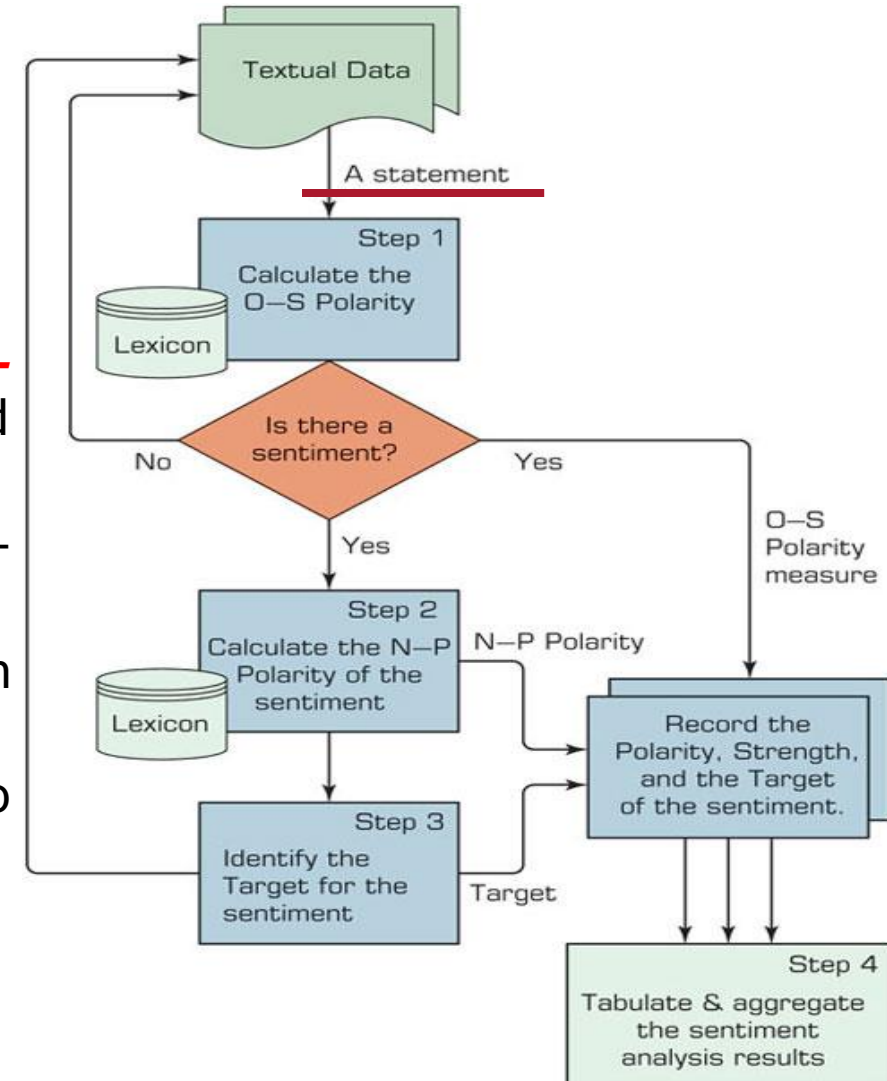
Sentiment Analysis

- Sentiment analysis is often referred to as opinion mining, subjectivity analysis, and appraisal extraction
- Sentiment analysis deals with the automatic extraction of ***opinions, feelings, and subjectivity*** in text,
- Sentiment analysis is trying to answer the question, "***What do people feel about a certain topic?***"
- ***Recommended Code:*** <https://github.com/RichardRivaldo/Sentiment-Analysis>

Sentiment Analysis Process

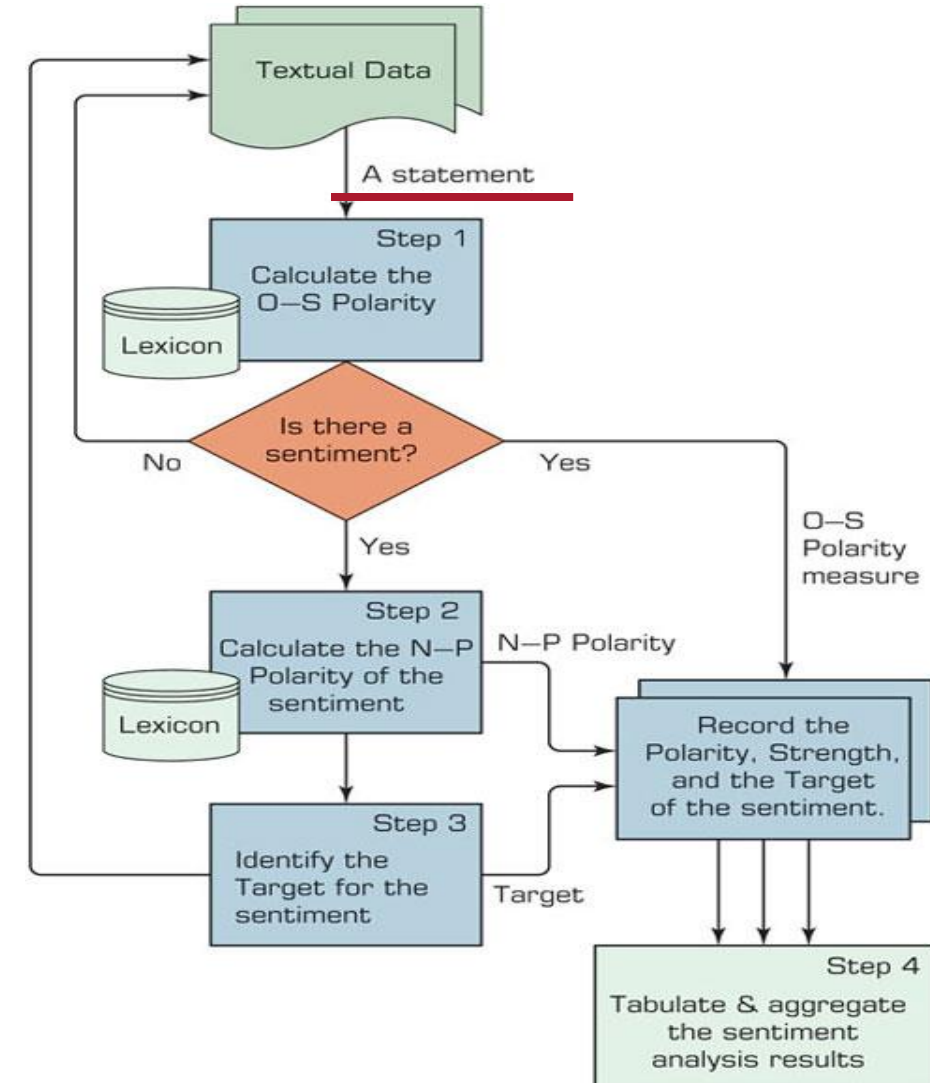
- **STEP 1: SENTIMENT DETECTION**

- Detect the objectivity by calculating the **Objectivity-Subjectivity [O-S] Polarity**, which can be represented with a numerical value ranging from 0 to 1.
- **The [O-S] Polarity** = (number of objective words - number of subjective words)/total number of words
- If the objectivity value is close to **1**, there is no opinion to mine (i.e., **it is a fact**)
- The process goes back and grabs the next text data to analyze.



Sentiment Analysis Process

- **STEP 2: N-P (NEGATIVE OR POSITIVE) POLARITY CLASSIFICATION**
 - Classify the opinion as falling under one of two opposing sentiment polarities
 - Polarity classification is ***the binary classification task of labeling an opinionated document as expressing either an overall positive or an overall negative opinion.***
 - In addition to the identification of N-P polarity, one identifies ***the strength of the sentiment*** (as opposed to just positive, it can be expressed as ***mildly, moderately, strongly, or very strongly positive***).



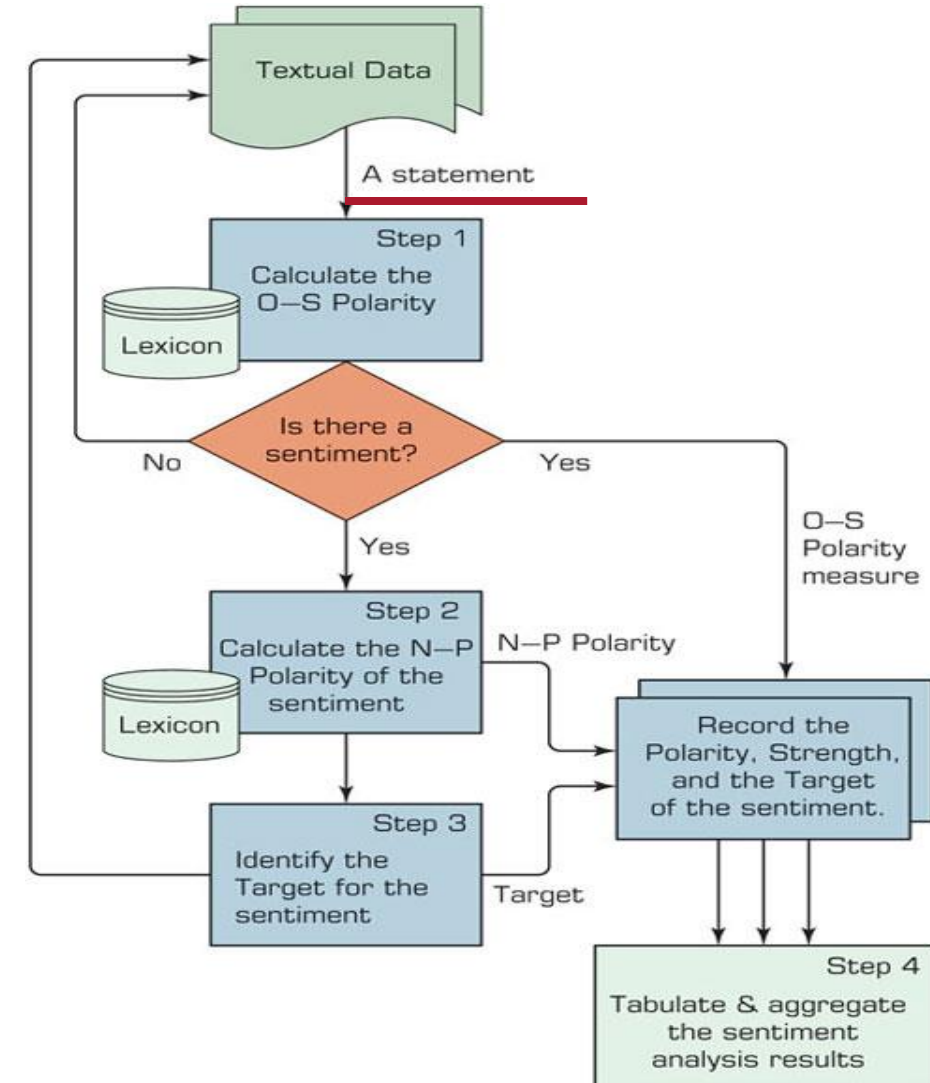
Sentiment Analysis Process

- **STEP 3: TARGET IDENTIFICATION**

- Accurately identify the target of the expressed sentiment (**e.g., a person, a product, an event**).

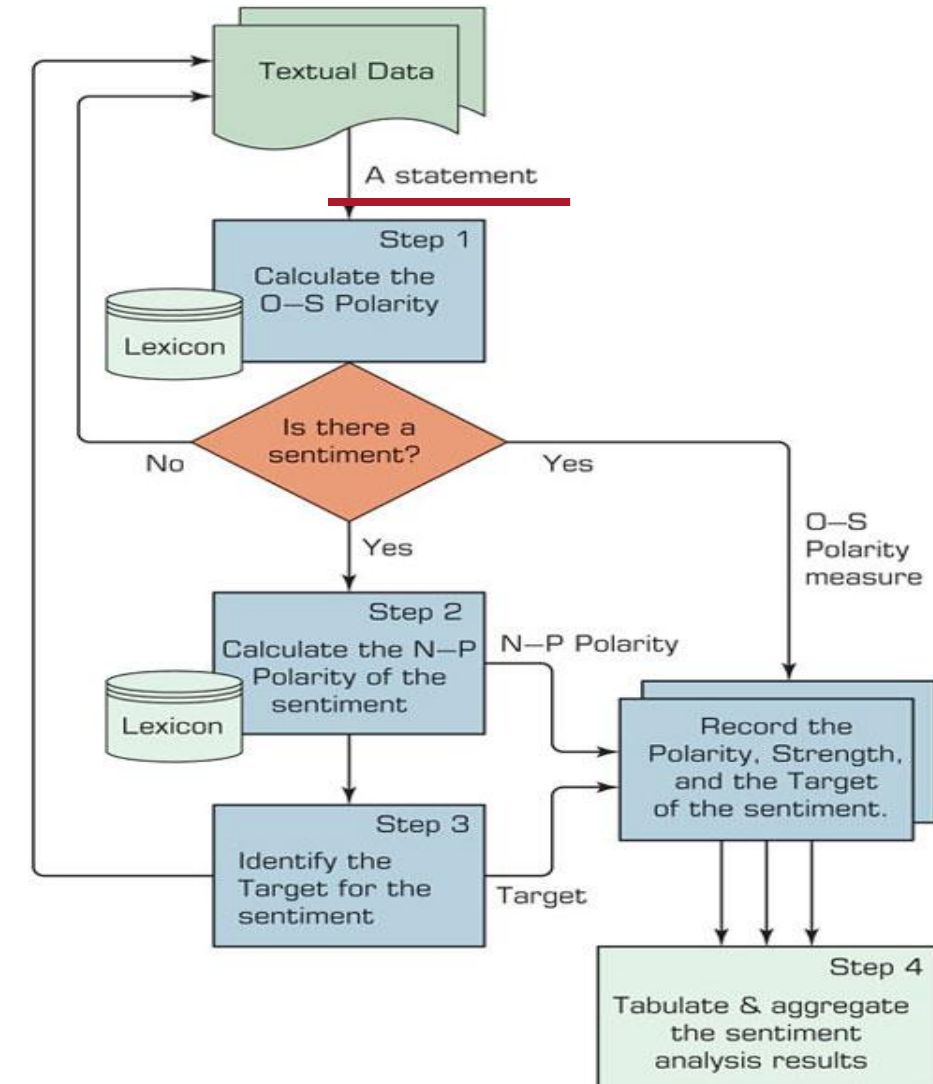
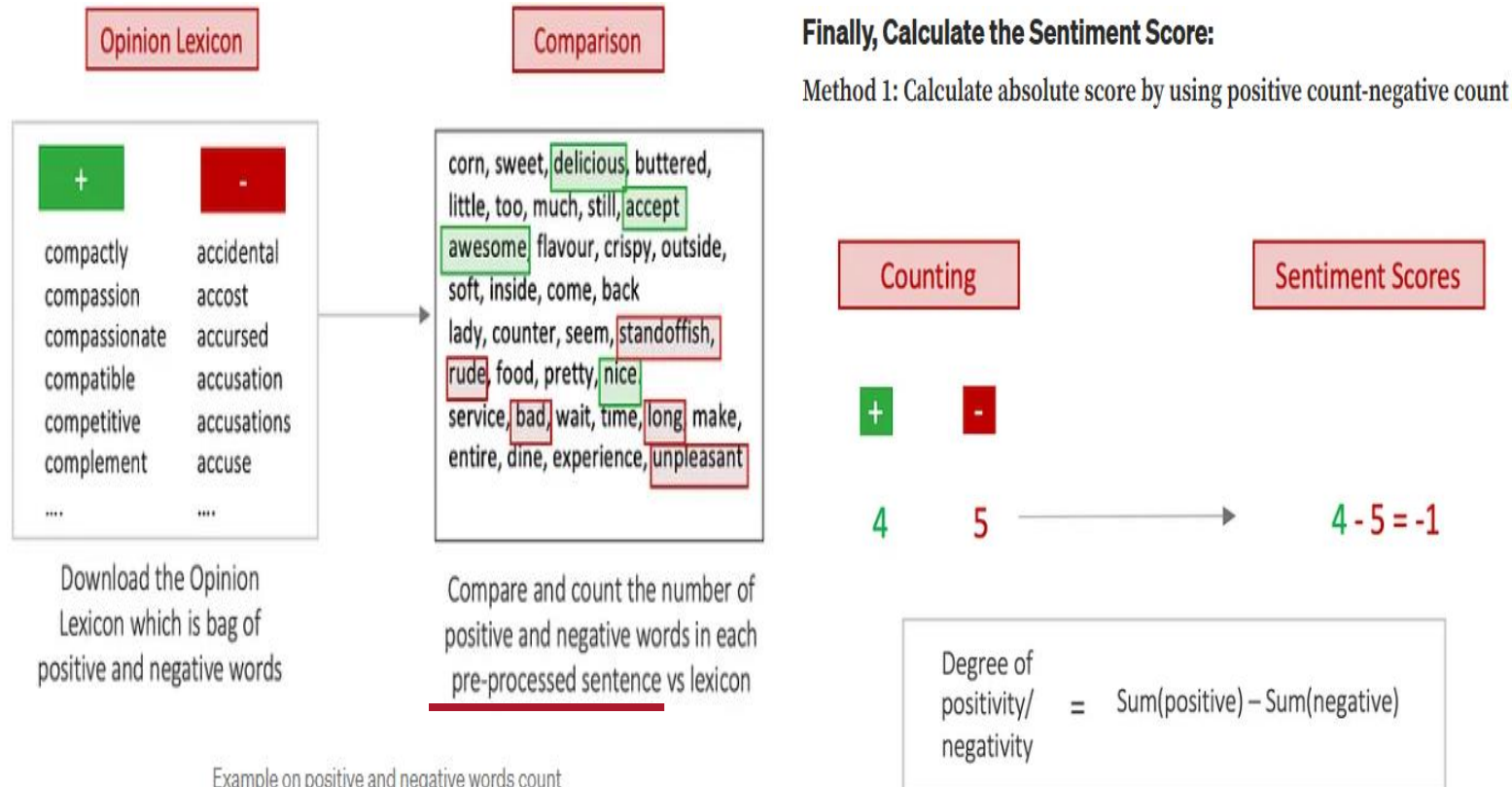
- **STEP 4: COLLECTION AND AGGREGATION**

- Once the sentiments of all text data points in the document have been identified and calculated, they are **aggregated and converted to a single sentiment measure for the entire document**.
- This aggregation could be as simple as **averaging the polarities and strengths** of all texts or as complex as **using semantic aggregation techniques** from NLP to identify the ultimate sentiment.



Two Methods for Polarity Identification

- 1. Using a **lexicon** as a reference library (developed either manually or automatically by an individual for a specific task or developed by an institution for general use).
 - WordNet (wordnet.princeton.edu)
 - SentiWordNet (<https://github.com/aesuli/SentiWordNet>)
 - WordNet-Affect (corpustext.com/reference/affect_wordnet.html)



Two Methods for Polarity Identification

- 1. Using **a lexicon** as a reference library (developed either manually or automatically by an individual for a specific task or developed by an institution for general use).
 - WordNet (wordnet.princeton.edu)
 - SentiWordNet (<https://github.com/aesuli/SentiWordNet>)
 - WordNet-Affect (corpus-text.com/reference/affect_wordnet.html)

Method 2: Normalise the score by the length of the review

Opinion Lexicon

Comparison

Counting

Sentiment Scores

+	-
compactly	accidental
compassion	accost
compassionate	accursed
compatible	accusation
competitive	accusations
complement	accuse
....	

corn, sweet, delicious, buttered,
 little, too, much, still, accept
 awesome flavour, crispy, outside,
 soft, inside, come, back
 lady, counter, seem, standoffish,
 rude, food, pretty, nice
 service, bad, wait, time, long make,
 entire, dine, experience, unpleasant

Words count: 35

4 5

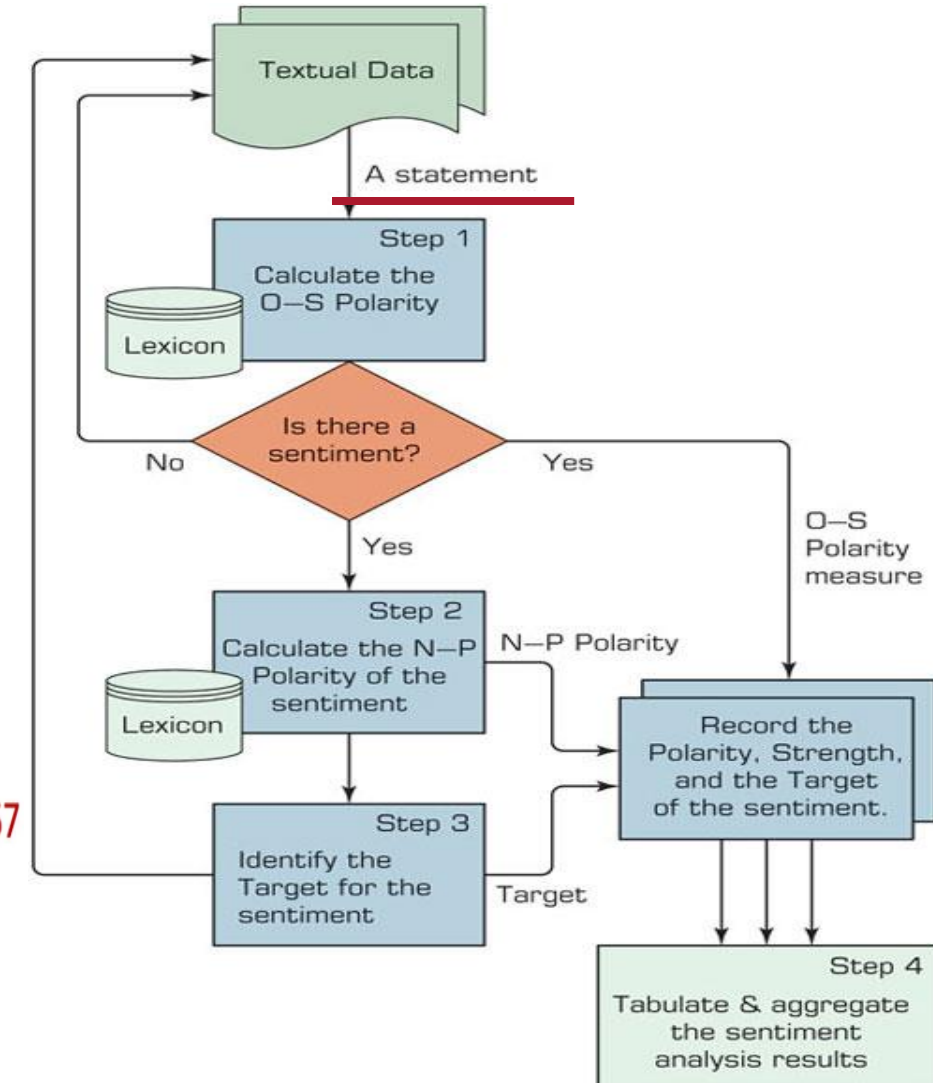
$(4 - 5)/35 = -0.02857$

$$\text{Degree of positivity/negativity} = \frac{\text{Sum(positive)} - \text{Sum(negative)}}{\text{Total word count}}$$

Download the Opinion Lexicon which is bag of positive and negative words

Compare and count the number of positive and negative words in each pre-processed sentence vs lexicon

Example on positive and negative words count



Two Methods for Polarity Identification

- 1. Using **a lexicon** as a reference library (developed either manually or automatically by an individual for a specific task or developed by an institution for general use).
 - WordNet (wordnet.princeton.edu)
 - SentiWordNet (<https://github.com/aesuli/SentiWordNet>)
 - WordNet-Affect (corpustext.com/reference/affect_wordnet.html)

Method 3: Calculate the ratio of positive and negative words count

Opinion Lexicon

Comparison

Counting

Sentiment Scores

+	-
compactly	accidental
compassion	accost
compassionate	accursed
compatible	accusation
competitive	accusations
complement	accuse
....	

corn, sweet, delicious, buttered,
 little, too, much, still, accept
 awesome, flavour, crispy, outside,
 soft, inside, come, back
 lady, counter, seem, standoffish,
 rude, food, pretty, nice
 service, bad, wait, time, long make,
 entire, dine, experience, unpleasant

+

-

4

5

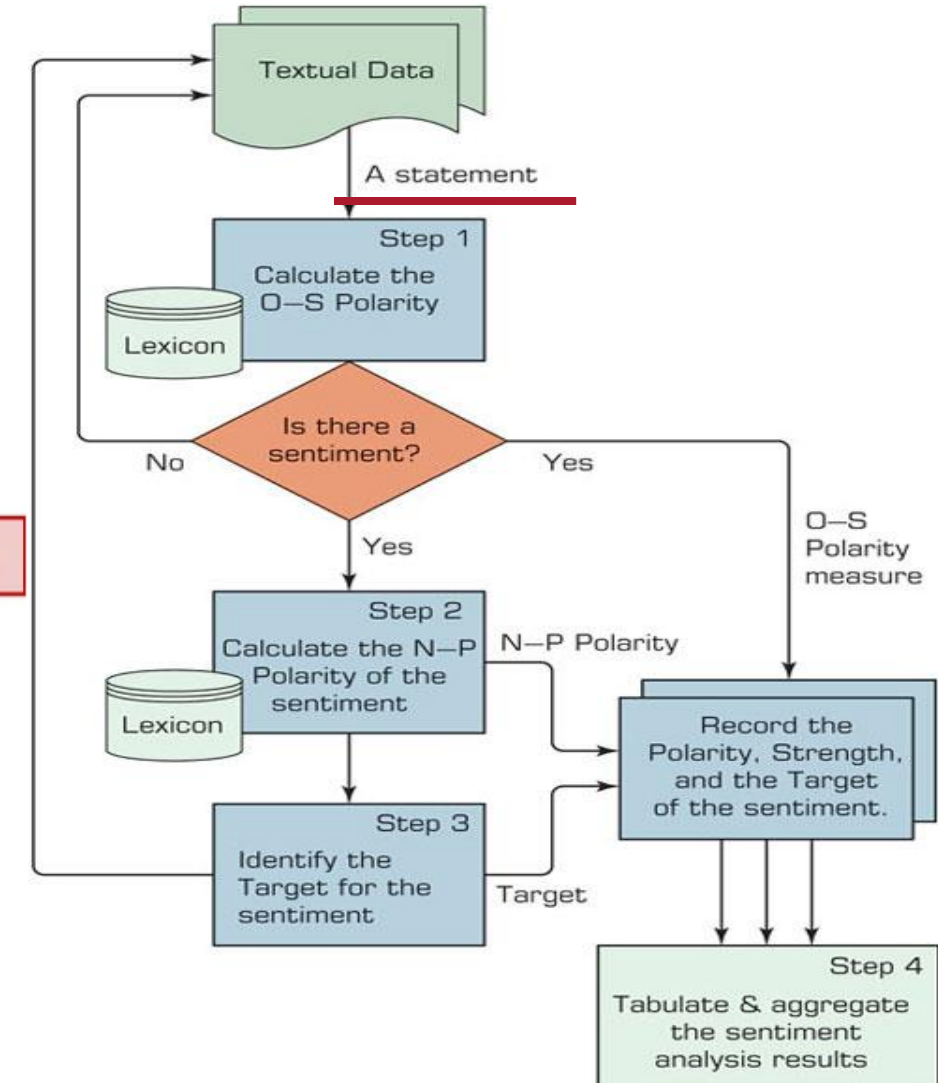
$4 / 5 + 1 = 0.67$

Download the Opinion Lexicon which is bag of positive and negative words

Compare and count the number of positive and negative words in each pre-processed sentence vs lexicon

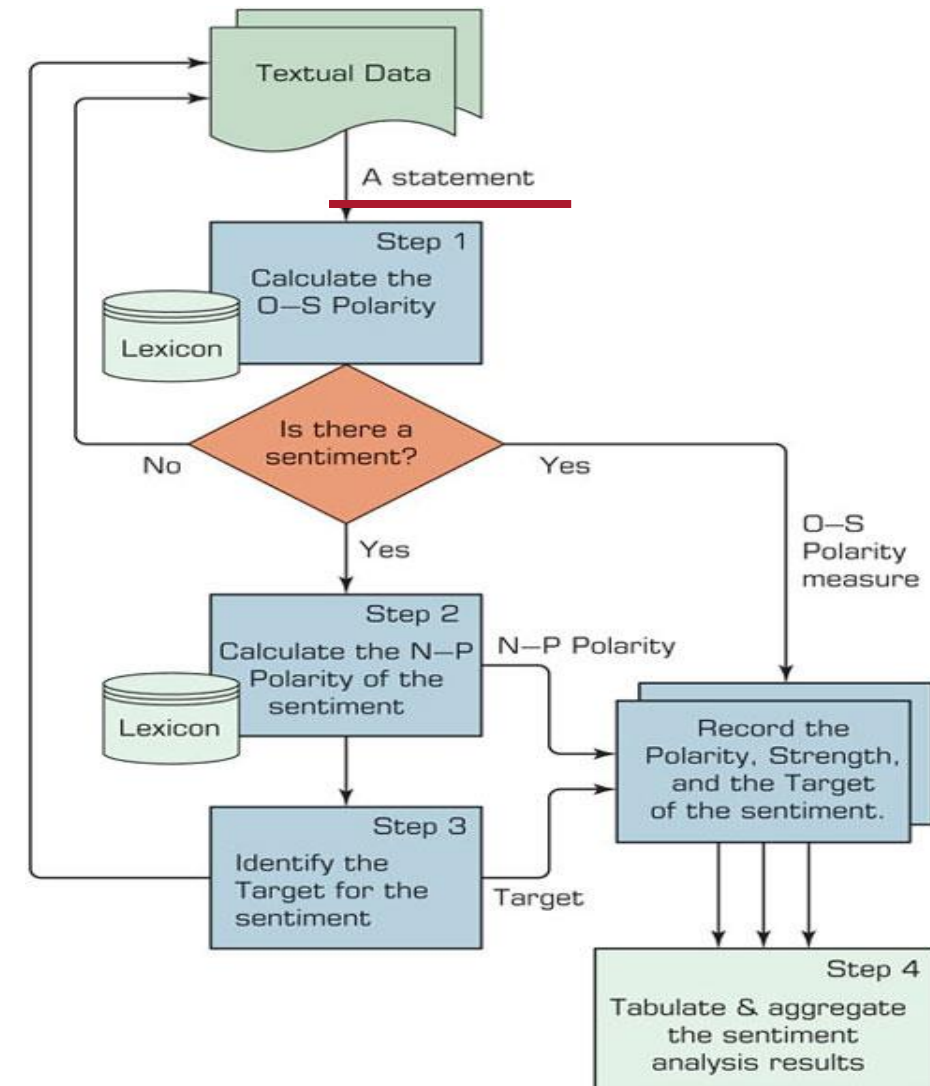
$$\text{Degree of positivity/negativity} = \frac{\text{Sum(positive)}}{\text{Sum(negative)+1}}$$

Example on positive and negative words count



Two Methods for Polarity Identification

- 2. Using **a collection of training documents** as the source of knowledge about the polarity of terms within a specific domain (i.e., inducing predictive models from opinionated textual documents).
 - Perform sentiment classification using statistical analysis and machine learning tools that take advantage of the vast resources of **labeled documents available**.
 - Once an already labeled textual data set has been obtained, **a variety of predictive modeling and other machine-learning algorithms** can be used to train sentiment classifiers.
 - Some of the **most popular algorithms** used for this task include artificial neural networks, support vector machines, k-nearest neighbor, Naïve Bayes, decision trees, and expectation maximization-based clustering.



Part-of-speech (POS) Tagging

- To categorize words by their **POS** is called **lexical category** or **tag**.
- The figure shows the 45 tags in the Penn Treebank (<https://catalog.ldc.upenn.edu/LDC99T42>).
- The task of assigning **a part of speech** to each word in a sentence is called **part-of-speech tagging**.
- One common model can be applied to the POS tagging task: **Hidden Markov Model (a stochastic technique)** and **Viterbi Algorithm that optimizes HMM**.



Tag	Word	Description	Tag	Word	Description
CC	<i>and</i>	Coordinating conjunction	PRP\$	<i>your</i>	Possessive pronoun
CD	<i>three</i>	Cardinal number	RB	<i>quickly</i>	Adverb
DT	<i>the</i>	Determiner	RBR	<i>quicker</i>	Adverb, comparative
EX	<i>there</i>	Existential there	RBS	<i>quickest</i>	Adverb, superlative
FW	<i>per se</i>	Foreign word	RP	<i>off</i>	Particle
IN	<i>of</i>	Preposition	SYM	<i>+</i>	Symbol
JJ	<i>purple</i>	Adjective	TO	<i>to</i>	to
JJR	<i>better</i>	Adjective, comparative	UH	<i>eureka</i>	Interjection
JJS	<i>best</i>	Adjective, superlative	VB	<i>talk</i>	Verb, base form
LS	<i>I</i>	List item marker	VBD	<i>talked</i>	Verb, past tense
MD	<i>should</i>	Modal	VBG	<i>talking</i>	Verb, gerund
NN	<i>kitten</i>	Noun, singular or mass	VCN	<i>talked</i>	Verb, past participle
NNS	<i>kittens</i>	Noun, plural	VBP	<i>talk</i>	Verb, non-3rd-sing
NNP	<i>Ali</i>	Proper noun, singular	VBZ	<i>talks</i>	Verb, 3rd-sing
NNPS	<i>Fords</i>	Proper noun, plural	WDT	<i>which</i>	Wh-determiner
PDT	<i>all</i>	Predeterminer	WP	<i>who</i>	Wh-pronoun
POS	<i>'s</i>	Possessive ending	WP\$	<i>whose</i>	Possessive wh-pronoun
PRP	<i>you</i>	Personal pronoun	WRB	<i>where</i>	Wh-adverb
\$	<i>\$</i>	Dollar sign	#	<i>#</i>	Pound sign
"	<i>'</i>	Left quote	"	<i>'</i>	Right quote
(	<i>[</i>	Left parenthesis	)	<i>]</i>	Right parenthesis
,	<i>,</i>	Comma	.	<i>!</i>	Sentence end
:	<i>;</i>	Mid-sentence punctuation			

Figure 23.1 Part-of-speech tags (with an example word for each tag) for the Penn Treebank corpus (?). Here “3rd-sing” is an abbreviation for “third person singular present tense.”

Grammar

- A **grammar** is a list of rules that define the set of all well-formed sentences in a language
- A **language** is the set of sentences that follow those rules.
- A **probabilistic context-free grammar** assigns a **probability** to each string and **context-free** means that any rule can be used in any context. Note:
 - The rules for a noun phrase at the beginning of a sentence are the same as for another noun phrase later in the sentence.
 - If the same phrase occurs in two locations, it must have the same probability each time.
 - We call the grammar for language ϵ_0 . For example,
 - $Adjs \rightarrow Adjective$ [0.8]
 - $| Adjective Adjs$ [0.2]
 - The syntactic category *Adjs* can consist of **either** a single Adjective with the probability 0.8 or of an Adjective followed by a string that constitutes an *Adjs.*, with the probability 0.2.

Grammar

S	$\rightarrow NP VP$	[0.90]	I + feel a breeze
	$ S Conj S$	[0.10]	I feel a breeze + and + It stinks
NP	$\rightarrow Pronoun$	[0.25]	I
	$ Name$	[0.10]	Ali
	$ Noun$	[0.10]	pits
	$ Article Noun$	[0.25]	the + wumpus
	$ Article Adjs Noun$	[0.05]	the + smelly dead + wumpus
	$ Digit Digit$	[0.05]	3 4
	$ NP PP$	[0.10]	the wumpus + in 1 3
	$ NP RelClause$	[0.05]	the wumpus + that is smelly
	$ NP Conj NP$	[0.05]	the wumpus + and + I
VP	$\rightarrow Verb$	[0.40]	stinks
	$ VP NP$	[0.35]	feel + a breeze
	$ VP Adjective$	[0.05]	smells + dead
	$ VP PP$	[0.10]	is + in 1 3
	$ VP Adverb$	[0.10]	go + ahead
$Adjs$	$\rightarrow Adjective$	[0.80]	smelly
	$ Adjective Adjs$	[0.20]	smelly + dead
PP	$\rightarrow Prep NP$	[1.00]	to + the east
$RelClause$	$\rightarrow RelPro VP$	[1.00]	that + is smelly

Figure 23.2 The grammar for $\mathcal{E}_0$, with example phrases for each rule. The syntactic categories are sentence (S), noun phrase (NP), verb phrase (VP), list of adjectives ($Adjs$), prepositional phrase (PP), and relative clause ($RelClause$).

Grammar

- The lexicon, or **a list of allowable words**, is defined below.
- Each of the lexical categories ends in ... to indicate that there are **other words in the category**.
- Nouns, names, verbs, adjectives, and adverbs are called **open classes**.
- Pronouns, relative pronouns, articles, preposition, and conjunctions are called **closed classes**.

<i>Noun</i>	→	stench [0.05] breeze [0.10] wumpus [0.15] pits [0.05] ...
<i>Verb</i>	→	is [0.10] feel [0.10] smells [0.10] stinks [0.05] ...
<i>Adjective</i>	→	right [0.10] dead [0.05] smelly [0.02] breezy [0.02] ...
<i>Adverb</i>	→	here [0.05] ahead [0.05] nearby [0.02] ...
<i>Pronoun</i>	→	me [0.10] you [0.03] I [0.10] it [0.10] ...
<i>RelPro</i>	→	that [0.40] which [0.15] who [0.20] whom [0.02] ...
<i>Name</i>	→	Ali [0.01] Bo [0.01] Boston [0.01] ...
<i>Article</i>	→	the [0.40] a [0.30] an [0.10] every [0.05] ...
<i>Prep</i>	→	to [0.20] in [0.10] on [0.05] near [0.10] ...
<i>Conj</i>	→	and [0.50] or [0.10] but [0.20] yet [0.02] ...
<i>Digit</i>	→	0 [0.20] 1 [0.20] 2 [0.20] 3 [0.20] 4 [0.20] ...

Figure 23.3 The lexicon for $\mathcal{E}_0$. *RelPro* is short for relative pronoun, *Prep* for preposition, and *Conj* for conjunction. The sum of the probabilities for each category is 1.

Complications of Real Natural Language

- **Quantification:** It is the kind of linguistic construction that specifies *the quantity of individuals* in the domain of discourse that satisfy given conditions.
- **Pragmatics:** It is a branch of linguistics concerned with the use of language in social contexts and the ways people produce and comprehend *meanings* through language.
- **Long-distance Dependencies:** It means any *dependency* that need *not hold* between strictly linearly adjacent words or morphemes.
- **Time and Tense:** English uses verb tenses (past, present, and future) to indicate the *relative time of an event*.
- **Ambiguity:** The *utterance* is unclear or confusing.

Natural Language Tasks

- **Speech Recognition:** The task of transforming spoken *sound into text*
 - Recurrent Neural Networks + Hidden Markov Models
 - Deep Neural Networks
- **Text-to-Speech:** The task of going from *text to sound*
 - Neural WaveNet System
- **Machine Translation:** The task of transforming text in *one language to another*.
 - Recurrent Neural Sequence-to-Sequence Models
 - Attention-focusing Mechanism of the Transformer Model
 - Hybrid Model
- **Information Extraction:** The task of *acquiring knowledge* by skimming a text and looking for occurrences of particular classes of objects and for relationships among them.
 - Regular Expression if the text is well structured.
 - TextRunner and Never-Ending Language Learning (NELL)
 - And more

Natural Language Tasks

- **Information Retrieval:** The task of *finding documents* that are relevant and important for a given query.
 - Google and Baidu
- **Question Answering:** The query is *a question*, and the response is *an actual answer*.
 - The START parser and question answerer
 - ASKMSR

Word Embeddings

- Encode a word into an input vector $\mathbf{x}$: **a one-hot vector** and **a word-embedded vector**.
- **A one-hot vector**: we encode the i th word in the dictionary with a 1 bit in the i th input position and a 0 in all the other positions.
 - If each word is represented with a vector of n -gram counts of all the phrases that the word appears in, the size of each vector of a word is **very large and sparse** – **most of the counts would be zero**.
- **A word-embedded vector** is a smaller-size vector with a few hundred dimensions.
 - **A low-dimensional, small, dense vector** represents a word.
 - Word embeddings are learned automatically from the data.
 - Each one is a vector of numbers, where the individual dimensions and their numeric values do not have discernible meanings.
 - The feature space has the property that similar words end up having similar vectors.

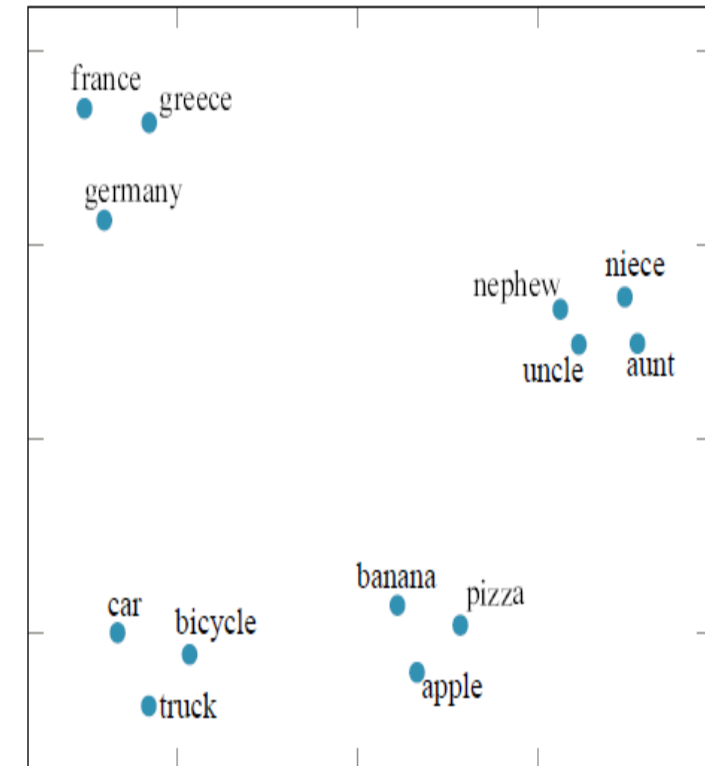


Figure 24.1 Word embedding vectors computed by the GloVe algorithm trained on 6 billion words of text. 100-dimensional word vectors are projected down onto two dimensions in this visualization. Similar words appear near each other.

Word Embeddings

- A word-embedded vector is a smaller-size vector with a few hundred dimensions.
- The word embedding vectors have **additional properties**.
- There is **no guarantee** that a particular word embedding algorithm run on a particular corpus will capture a particular semantic relationship.
- The commonly used vector dictionaries include **Word2Vec**, **GloVe (Global Vectors)**, and **FastText**.

A	B	C	$D = C + (B - A)$	Relationship
Athens	Greece	Oslo	Norway	Capital
Astana	Kazakhstan	Harare	Zimbabwe	Capital
Angola	kwanza	Iran	rial	Currency
copper	Cu	gold	Au	Atomic Symbol
Microsoft	Windows	Google	Android	Operating System
New York	New York Times	Baltimore	Baltimore Sun	Newspaper
Berlusconi	Silvio	Obama	Barack	First name
Switzerland	Swiss	Cambodia	Cambodian	Nationality
Einstein	scientist	Picasso	painter	Occupation
brother	sister	grandson	granddaughter	Family Relation
Chicago	Illinois	Stockton	California	State
possibly	impossibly	ethical	unethical	Negative
mouse	mice	dollar	dollars	Plural
easy	easiest	lucky	luckiest	Superlative
walking	walked	swimming	swam	Past tense

Figure 24.2 A word embedding model can sometimes answer the question “A is to B as C is to [what]?” with vector arithmetic: given the word embedding vectors for the words A, B, and C, compute the vector $D = C + (B - A)$ and look up the word that is closest to D. (The answers in column D were computed automatically by the model. The descriptions in the “Relationship” column were added by hand.) Adapted from ? (? , ?).

Word Embeddings

- Given a corpus of sentences with POS tags, we learn the parameters for the word embeddings and the **POS** tagger simultaneously. The process works as follows:
 - STEP 1:** Choose the width w , an odd number of words, for the prediction window to be used to tag each word.
 - A typical value is $w = 5$, meaning that the tag is predicted based on the word plus the two words to the left and the two words to the right.
 - Each window produces one training example consisting of the w words as input and the POS category of the middle word as output.
 - STEP 2:** Create a **vocabulary** of all of the unique word tokens that occur **more than, e.g., 10 times** in the training data. Denote **the total number of words** in the vocabulary as v .
 - STEP 3:** Sort this vocabulary in an alphabetical order.
 - STEP 4:** Choose a value d as **the size of each word embedding vector**.

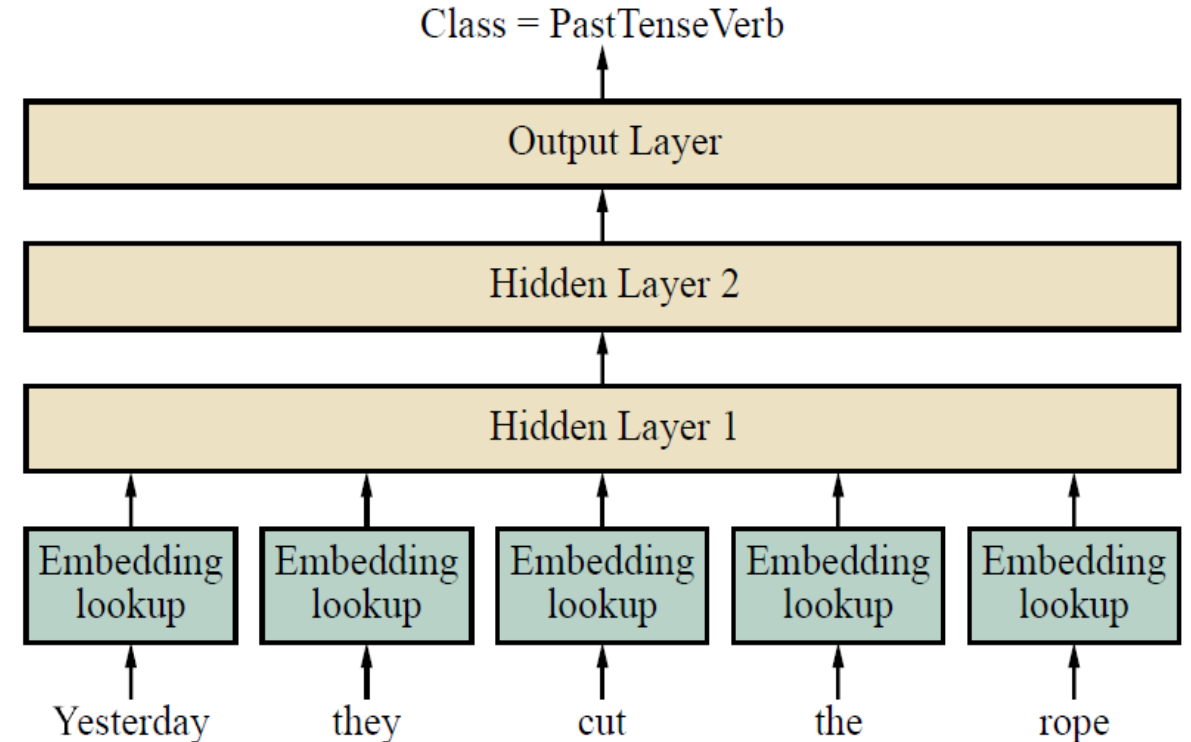


Figure 24.3 | Feedforward part-of-speech tagging model. This model takes a 5-word window as input and predicts the tag of the word in the middle—here, *cut*. The model is able to account for word position because each of the 5 input embeddings is multiplied by a different part of the first hidden layer. The parameter values for the word embeddings and for the three layers are all learned simultaneously during training.

Word Embeddings

- Given a corpus of sentences with POS tags, we learn the parameters for the word embeddings and the POS tagger simultaneously. The process works as follows:

- STEP 5:** Create a new ***v-by-d*** weight matrix called ***E***, i.e., the word embedding matrix.
 - Row ***i*** of ***E*** is the word embedding of the *i*th word in the vocabulary.
 - Initialize ***E*** randomly or from pretrained vectors.

- STEP 6:** Set up a neural network that outputs a POS label.
 - The 1st layer will consist of ***w*** copies of the embedding matrix.
 - The two additional hidden layers, ***z₁*** and ***z₂*** with weight matrices ***W₁*** and ***W₂***, followed by a softmax layer yielding an output probability distribution $\hat{y}$ over the possible POS categories for the middle word: $\mathbf{z}_1 = \sigma(\mathbf{W}_1 \mathbf{x})$, $\mathbf{z}_2 = \sigma(\mathbf{W}_2 \mathbf{z}_1)$, and $\hat{y} = \text{softmax}(\mathbf{W}_{\text{out}} \mathbf{z}_2)$.

- STEP 7: Concatenate** the embedding vectors to encode a sequence of ***w*** words into an input vector. The result is a real-valued input vector of ***x*** of length ***wd***.

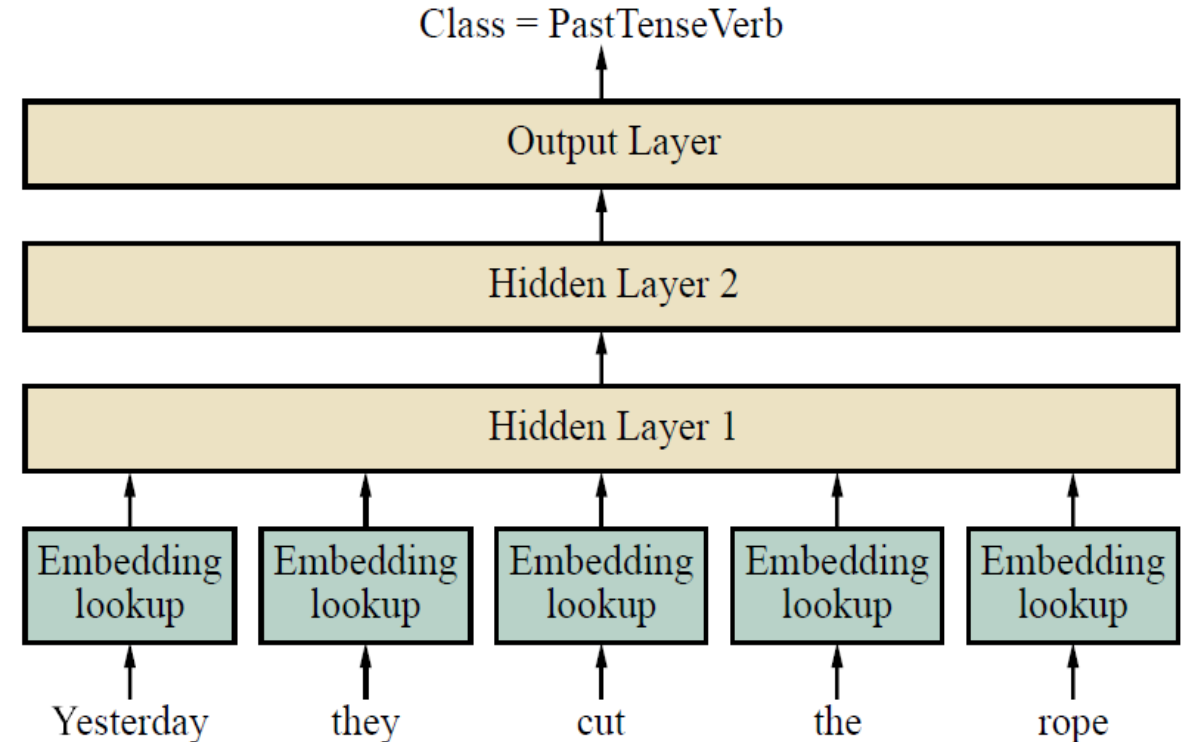


Figure 24.3 | Feedforward part-of-speech tagging model. This model takes a 5-word window as input and predicts the tag of the word in the middle—here, *cut*. The model is able to account for word position because each of the 5 input embeddings is multiplied by a different part of the first hidden layer. The parameter values for the word embeddings and for the three layers are all learned simultaneously during training.

Word Embeddings

- Given a corpus of sentences with POS tags, we learn the parameters for the word embeddings and the POS tagger simultaneously. The process works as follows:
 - STEP 8:** Train the weights E and the other weight matrices W_1 , W_2 , and W_{out} using gradient descent.
- A Character-level Model:** the input is a sequence of characters, each encoded as a one-hot vector.
 - Such a model has to learn how characters come together to form words.
 - The majority of work in NLP sticks with word-level rather than character-level encodings.

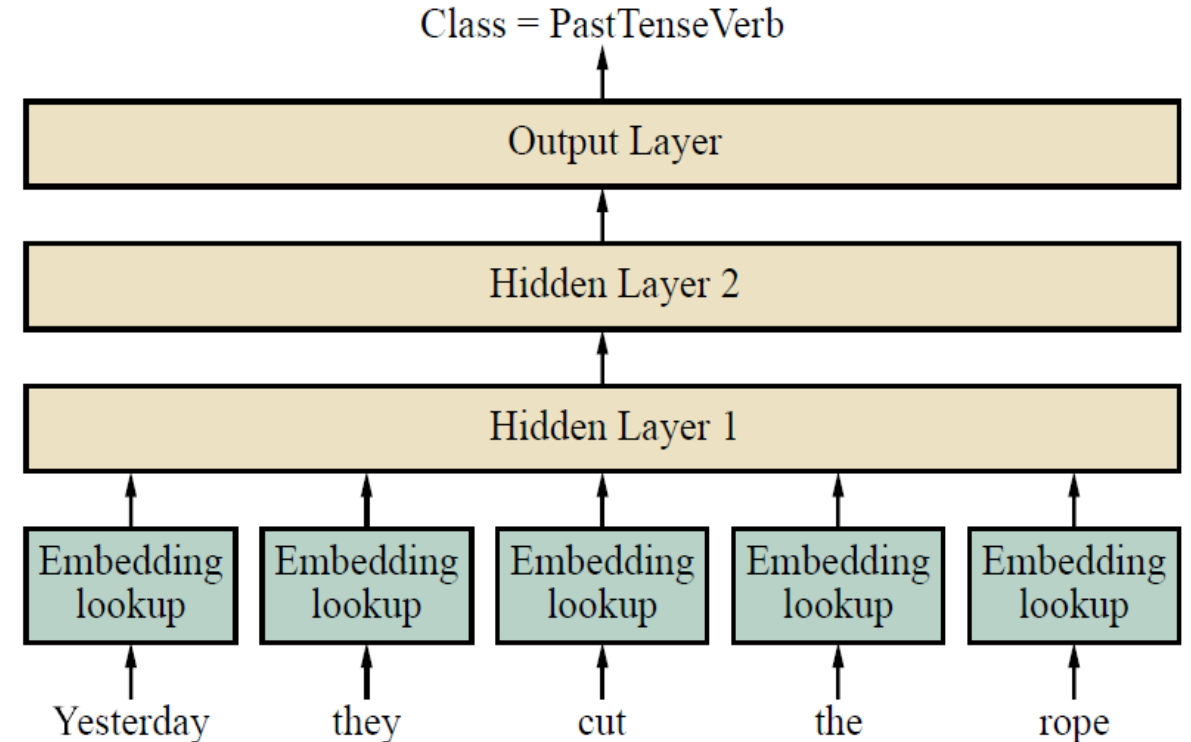


Figure 24.3 Feedforward part-of-speech tagging model. This model takes a 5-word window as input and predicts the tag of the word in the middle—here, cut. The model is able to account for word position because each of the 5 input embeddings is multiplied by a different part of the first hidden layer. The parameter values for the word embeddings and for the three layers are all learned simultaneously during training.

Language Models with Recurrent Neural Networks

- A language model is a probability distribution over sequences of words. **It predicts the next word in a text given all the previous words.**
- In an RNN language model, **each input word is encoded as a word embedding vector, $\mathbf{x}_t$.** There is **a hidden layer $\mathbf{z}_t$** which gets passed as input from one time step to the next. The output $\mathbf{y}_t$ will be **a softmax probability distribution** over the possible values of the next word in the sentence.
- **Theory:** There is no limit to how far back in the input the model can look. Each update of the hidden layer $\mathbf{z}_t$ has access to both the current input word $\mathbf{x}_t$ and the previous hidden layer $\mathbf{z}_{t-1}$, which means **that information about any word in the input can be kept in the hidden layer indefinitely, copied over from one time step to the next.**
- **Reality:** There is a limited amount of storage in $\mathbf{z}$, so it can't remember everything about all the previous words.

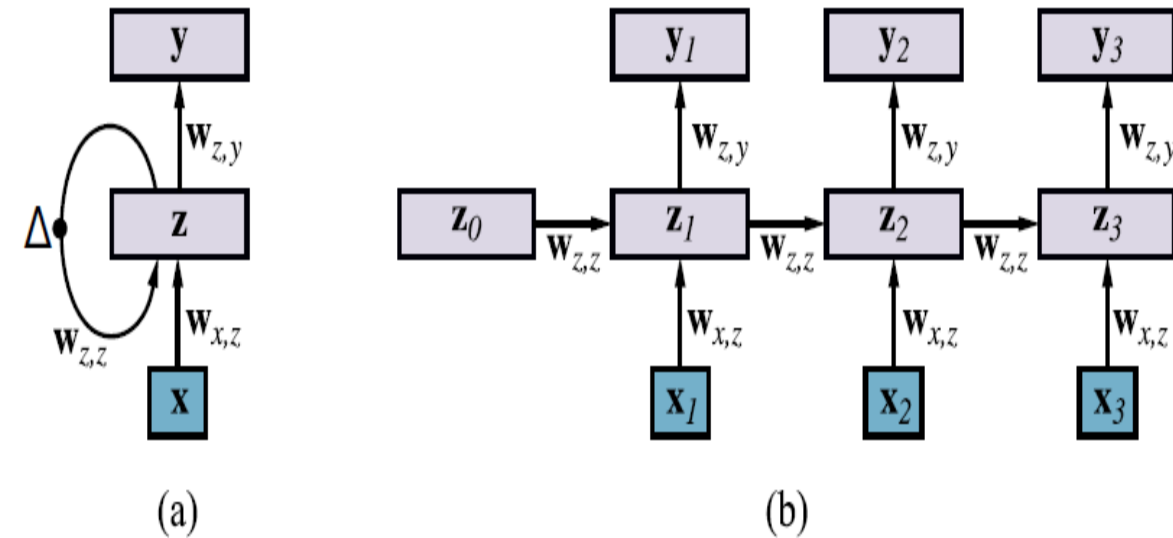
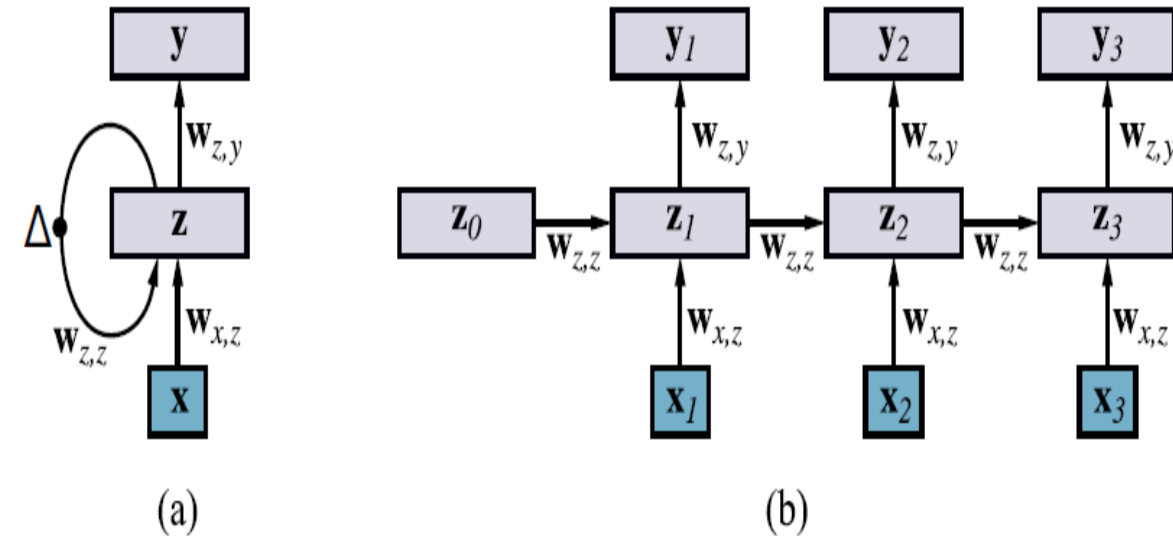


Figure 24.4 (a) Schematic diagram of an RNN where the hidden layer $\mathbf{z}$ has recurrent connections; the Δ symbol indicates a delay. Each input $\mathbf{x}$ is the word embedding vector of the next word in the sentence. Each output $\mathbf{y}$ is the output for that time step. (b) The same network unrolled over three timesteps to create a feedforward network. Note that the weights are shared across all timesteps.

Language Models with Recurrent Neural Networks

- **To train an RNN language model**, the inputs, $\mathbf{x}_t$, are the words in a training corpus of text, and the observed outputs are the same words offset by 1.
 - For the training text “hello world”, the first input $\mathbf{x}_1$ is the word embedding for “hello” and the first output $\mathbf{y}_1$ is the word embedding for “world”.
 - We compute the difference between the observed output and the actual output computed by the network, and back-propagate through time, taking care to keep the weights the same for all time steps.
- **Once the model has been trained, we can use it to generate random text.**
 - We give the model an initial input word $\mathbf{x}_1$, from which it will produce an output $\mathbf{y}_1$ which is a softmax probability distribution over words.
 - We sample a single word from the distribution, record the word as the output for time t , and feed it back in as the next input word $\mathbf{x}_2$.



Classification with Recurrent Neural Networks

- Training an RNN to do classification, the training data will require labels.
- For classification, we look at **not only the previous words but also look ahead** in the sentence.
- To capture the context on the right, we can use a bidirectional RNN, which concatenates a separate right-to-left model onto the left-to-right model.
- An example of using a bidirectional RNN for POS tagging:
 - $\mathbf{z}_t$ will be the hidden vector of the last layer.
 - $\mathbf{z}_t$ is taken to be the concatenation of vectors from the left-to-right and right-to-left.

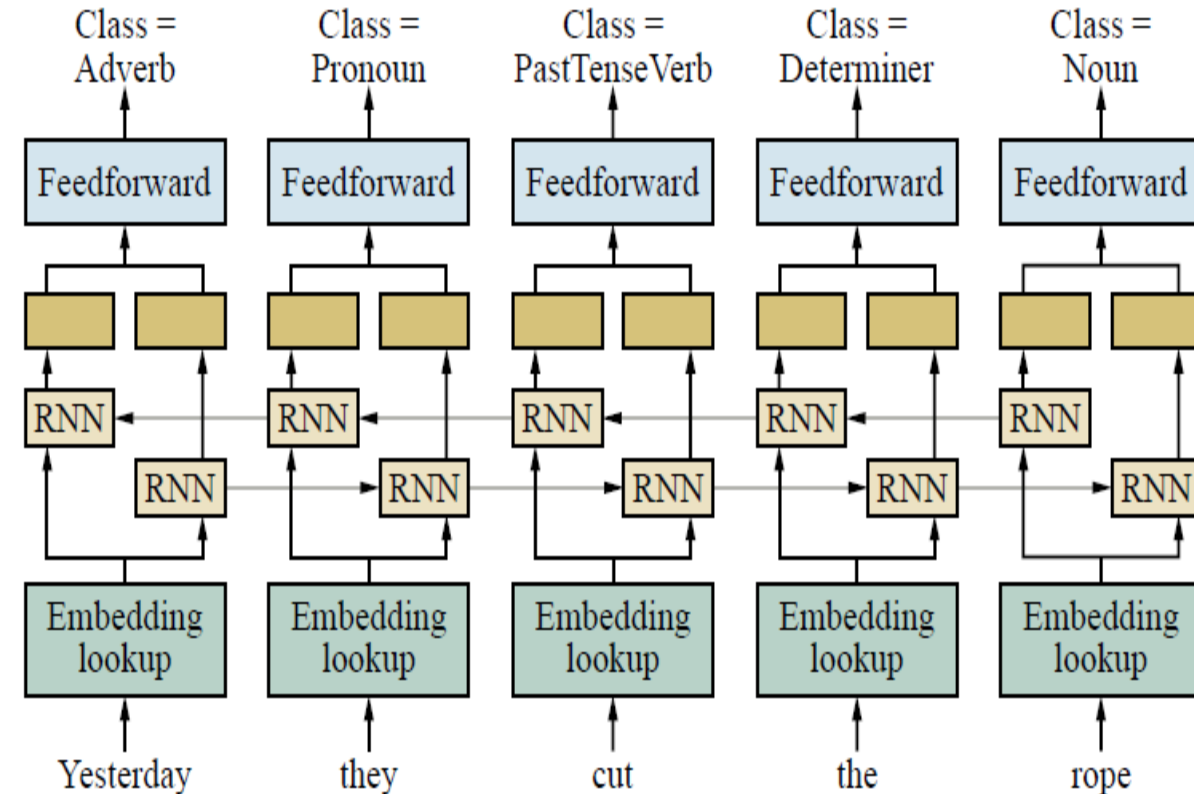


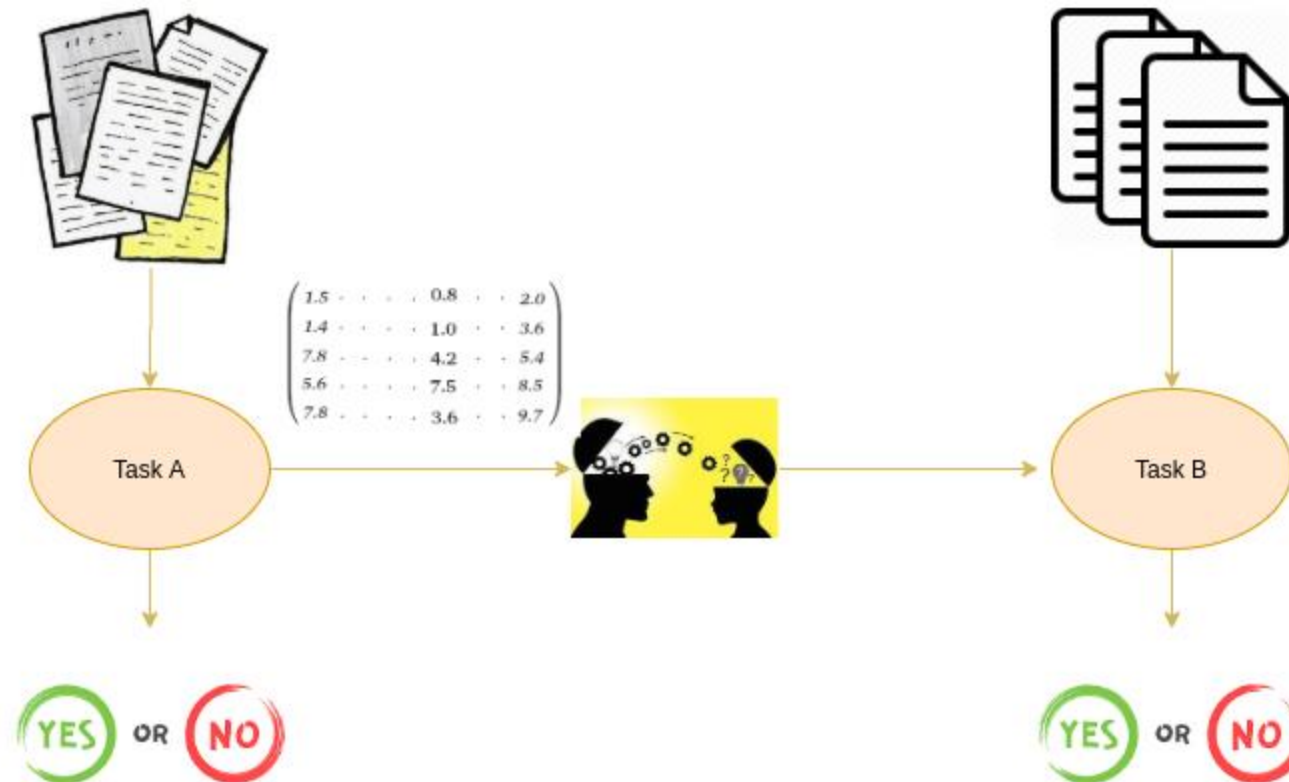
Figure 24.5 A bidirectional RNN network for POS tagging.

Pretraining and Transfer Learning

- For natural language, text is often unlabeled due to the difficulty of labeling and due to the abundance of text.
- It requires extensive training to annotate a sentence with POS tags.
- Pre-training and Transfer Learning: we use a large amount of shared general domain language data to train an initial version of an NLP model.
- We can use a smaller amount of domain-specific data to refine the pretrained model.

Pretrained Word Embeddings

- Pretrained Word Embeddings are **the embeddings learned in one task that are used for solving another similar task**.
- These embeddings are trained on large datasets, saved, and then used for solving other tasks. That's why pretrained word embeddings are a form of Transfer Learning.



The Top 10 NLP Tools

1. [MonkeyLearn](#) | NLP made simple
2. [Aylien](#) | Leveraging news content with NLP
3. [IBM Watson](#) | A pioneer AI platform for businesses
4. [Google Cloud NLP API](#) | Google technology applied to NLP
5. [Amazon Comprehend](#) | An AWS service to get insights from text
6. [NLTK](#) | The most popular Python library
7. [Stanford Core NLP](#) | Stanford's fast and robust toolkit
8. [TextBlob](#) | An intuitive interface for NLTK
9. [SpaCy](#) | Super-fast library for advanced NLP tasks
10. [GenSim](#) | State-of-the-art topic modeling

End of Material for CS 534!

